- **Week 1** Exercises Solutions :

Sequence 1 — RAW Hazards with Forwarding

	1	2	3	4	5	6	7
add $s0,$t0,$t1	IF	ID	EX	MEM	WB		
sub $s1,$s0,$t2		IF	ID	EX	MEM	WB	
and $s2,$s1,$s0			IF	ID	EX	MEM	WB

1. Forwarding is active in

- Cycle 4 : EX/MEM → EX (I1 result [$s0] is forwarded to EX of Cycle 4)
- Cycle 5 : EX/MEM → EX (I2 result [$s1] is forwarded to EX of Cycle 5)

Sequence 2 — Load-Use Hazard with Stall

	1	2	3	4	5	6	7	8
lw $s0,0($t0)	IF	ID	EX	MEM	WB			
add $s1,$s0,$t1		IF	ID	STALL	EX	MEM	WB	
sub $s2,$s1,$t2			IF		ID	EX	MEM	WB

- Cycle 4 : a stall is introduced. It inserts bubble in I2 & I3 EX ,holds IF/ID stages.
- Cycle 5 : Forwarding is active in cycle 5 MEM/WB → EX
- Cycle 6 : Forwarding is active in cycle 6 MEM/WB → EX

- Forwarding won't help because the desired data from the lw instruction will not physically exist until it is read from the Data Memory at the end of stage 4 (MEM). However, the add instruction needs that data at the beginning of its EX stage (stage 3). This is physically impossible; we cannot forward data backwards in time.

Sequence 3 — Control Hazard with Taken Branch

	C1	C2	C3	C4	C5	C6	C7	C8
beq $t0,$t1,target	IF	ID						
add $s0,$s1,$s2		IF	FLUSH					
sub $s3,$s4,$s5			IF	FLUSH				
or $s6,$s7,$t0				IF	ID	EX	MEM	WB

- Branch resolved at C2 ID.
- Total instruction Flushed : 2
- Branches penalty of 2 cycles as the pipeline has already fetched two wrong instructions by the time correct path is fetched.

PART B

1. A **structural hazard** happens when two instructions need the same hardware at the same time. For example, if instruction memory and data memory are shared, an lw instruction accessing memory can conflict with the next instruction fetch. E.g. part 1 Sequence 2.

A **data hazard** happens when one instruction depends on the result of a previous instruction that has not finished yet. The second instruction may read the wrong value if the pipeline does not wait or forward data.
E.g.
add $t0, $t1, $t2
sub $t3, $t0, $t4

A **control hazard** occurs because of branch or jump instructions where the processor does not yet know which instruction should execute next. The wrong instructions may get fetched before the branch decision is made.
E.g. part A seq. 3..

2. For N independent add instructions with full forwarding, there are no stalls after the pipeline fills. So the minimum CPI is approximately 1 whereas for N lw instruc. There is one stall cycle per pair instruc. Therefore CPI is 2.

3. In a single-cycle MIPS processor, only one instruction can complete in one clock cycle because the whole instruction must go through fetch, decode, execute, memory, and writeback in the same cycle. So even if the clock becomes faster, the processor still finishes only one instruction per cycle. Therefore acc. To iron law for single cycle CPU ,CPI is always 1 so IPC = 1.

4. If branch resolution is moved from the EX stage to the ID stage, extra hardware is needed in the ID stage. This includes:

- a comparator to check branch conditions,
- forwarding logic into ID,
- and branch target calculation hardware.

The advantage is that the branch decision is made earlier. On a branch misprediction, the penalty reduces from about 2 cycles to 1 cycle because fewer wrong instructions are fetched. On a correct prediction, there is no penalty.

5. RAW hazard fixed by forwarding only.

```
 add $t0, $t1, $t2
sub $t3, $t0, $t4
and $t5, $t3, $t6
```

RAW hazard that requires a stall

```
lw $t0, 0($t1)
add $t2, $t0, $t3
```

6. In a strict in-order, single-issue MIPS pipeline, instructions always read operands before later instructions write results, and writes happen in program order. Because of this:

- **WAR (Write After Read)** hazards cannot occur.
- **WAW (Write After Write)** hazards also cannot occur.

In an out-of-order processor, instructions may finish in a different order. A later instruction could write a register before an earlier instruction reads or writes it, creating WAR or WAW hazards. That is why advanced CPUs need techniques like register renaming to avoid these problems.